

SQL : Structured Query Language

MySQL(database) - SQL (Language)

- **Database:** Collection of data in a format that can be easily accessed(Digital format)
- **DBMS:** Database Management System - software application used to manage databases

Types of Databases

Relational (SQL)	Non-relational (NoSQL)
Data stored in tables	Data not stored in tables Uses other query languages
Eg MySQL, Oracle, PostgreSQL	Eg MongoDB, Cassandra

SQL Basics

- **SQL:** Structured Query Language
- Programming language used to interact with relational databases
- Used to perform **CRUD operations**:
 - Create
 - Read
 - Update
 - Delete

Data Types

Common SQL Data Types

- **CHAR(size):** Fixed-length string
- **VARCHAR(size):** Variable-length string
- **INT:** Integer numbers
- **DATE:** Date values YYYY-MM-DD
- **FLOAT:** decimal values
- **BOOLEAN:** True/False values

Table in SQL

Column: design (schema) vertical

Rows: tuple (horizontal)

SQL Commands

Types of SQL Commands

1. DDL (Data Definition Language)

- a. CREATE, ALTER, RENAME, TRUNCATE, DROP

2. DML (Data Manipulation Language)

- a. SELECT, INSERT, UPDATE, DELETE

3. DCL (Data Control Language)

- a. GRANT, REVOKE (permissions to users)

4. DQL (Data Query Language)

- a. SELECT

5. TCL (Transaction Control Language)

- a. START TRANSACTION, COMMIT, ROLLBACK

Database Operations

CREATE DATABASE db_name;

CREATE DATABASE IF NOT EXISTS db_name;

DROP DATABASE db_name;

DROP DATABASE IF EXISTS db_name;

SHOW DATABASES;(Show all databases)

USE db_name;

SHOW TABLES;(Show all tables in current database)

Table Operations

Creating Tables

```
CREATE TABLE table_name (  
    column_name1 datatype constraint,  
    column_name2 datatype constraint,  
    column_name3 datatype constraint  
);
```

Viewing Table Data

-- Select all columns

```
SELECT * FROM table_name;
```

-- Select specific columns

```
SELECT col1, col2 FROM table_name;
```

Inserting Data

```
INSERT INTO table_name (colname1, colname2)  
VALUES
```

```
    (col1_v1, col2_v1),
```

```
    (col1_v2, col2_v2);
```

Updating Data: SET SQL_SAFE_UPDATES = 0;

```
UPDATE table_name
```

```
SET col1 = val1, col2 = val2
```

```
WHERE condition;
```

Deleting Data

```
DELETE FROM table_name
```

```
WHERE condition;
```

Altering Tables (alter to change the schema)

-- Add column

```
ALTER TABLE table_name
```

```
ADD COLUMN column_name datatype constraint;
```

-- Drop column

```
ALTER TABLE table_name
```

```
DROP COLUMN column_name;
```

-- Rename table

```
ALTER TABLE table_name
```

```
RENAME TO new_table_name;
```

-- Modify column datatype/constraint

```
ALTER TABLE table_name
```

```
MODIFY col_name new_datatype new_constraint;
```

-- Rename column

```
ALTER TABLE table_name
```

```
CHANGE COLUMN old_name new_name new_datatype new_constraint;
```

Truncating Tables

-- Delete all data from table (keeps structure)

```
TRUNCATE TABLE table_name;
```

Keys and Constraints

Keys

1. Primary Key (PK)

- Uniquely identifies each row in a table
- Only 1 PK per table
- Should be NOT NULL
- Automatically UNIQUE

2. Foreign Key (FK)

- References primary key in another table
- Can have duplicate and null values
- Can have multiple FKs in a table

Constraints: rule for data in the table

- NOT NULL:** Column cannot have null values
- UNIQUE:** All values in column must be different
- PRIMARY KEY:** Makes column unique and not null
- FOREIGN KEY:** Prevents actions that destroy links between tables
- DEFAULT:** Sets default value for a column
- CHECK:** Limits values allowed in a column

Use case

- NOT NULL:** `user_id INT NOT NULL.`
- UNIQUE:** `user_id INT UNIQUE`
- PRIMARY KEY:** `user_id INT PRIMARY KEY` - Combines UNIQUE + NOT NULL, main table identifier
- FOREIGN KEY:** `department_id INT FOREIGN KEY REFERENCES departments(dept_id)` - Links to another table
- DEFAULT:** `status VARCHAR(20) DEFAULT 'Active'` - Auto-fills 'Active' if no value provided
- CHECK:** `age INT CHECK (age >= 18)` - Only allows ages 18 and above

Cascading for Foreign Keys

- On Delete Cascade: Deletes child rows when parent is deleted
- On Update Cascade: Updates child rows when parent is updated

Data Retrieval (SELECT)

Basic SELECT Syntax

`SELECT col1, col2 FROM table_name;`

`SELECT * FROM table_name;` -- Select all columns

WHERE Clause: to define conditions

```
SELECT col1, col2  
FROM table_name  
WHERE conditions;
```

Operators and Clauses

Types of Operators

1. **Arithmetic Operators:** +, -, *, /, %
2. **Comparison Operators:** =, !=, >, >=, <, <=
3. **Logical Operators:** AND, OR, NOT, IN, BETWEEN, LIKE
4. **Bitwise Operators:** & (AND), | (OR)

Bitwise AND (&)

A	B	A & B
0	0	0
0	1	0
1	0	0
1	1	1

Bitwise OR (|)

A	B	A B
0	0	0
0	1	1
1	0	1
1	1	1

Logical Operators

-- AND: Both conditions must be true

```
SELECT *  
FROM table_name  
WHERE condition1 AND condition2;
```

-- OR: At least one condition must be true

```
SELECT *  
FROM table_name
```

WHERE condition1 OR condition2;

-- NOT: Negates the condition

```
SELECT *  
FROM table_name  
WHERE NOT condition;
```

-- IN: Matches any value in the list

```
SELECT *  
FROM table_name  
WHERE col_name IN (value1, value2, value3);
```

-- BETWEEN: Selects values within a range

```
SELECT *  
FROM table_name  
WHERE col_name BETWEEN value1 AND value2;
```

Other Important Clauses

LIMIT Clause

-- Limits number of rows returned

```
SELECT col1, col2 FROM table_name  
LIMIT number;
```

ORDER BY Clause

-- Sort in ascending order (default)

```
SELECT col1, col2  
FROM table_name  
ORDER BY col_name ASC;
```

-- Sort in descending order

```
SELECT col1, col2  
FROM table_name  
ORDER BY col_name DESC;
```

Aggregate Functions: perform a calculation on a set of values, and return a single value.

- **COUNT():** Counts number of rows
- **MAX():** Returns maximum value
- **MIN():** Returns minimum value
- **SUM():** Returns sum of values
- **AVG():** Returns average value

-- Examples

```
SELECT COUNT(*) FROM table_name;
```

```
SELECT AVG(marks) FROM students;  
SELECT MAX(marks) FROM students;
```

GROUP BY Clause

- Groups rows with same values into summary rows
- Usually used with aggregate functions

– Count students in each city
SELECT city, COUNT(*)
FROM students
GROUP BY city;

HAVING Clause

- Similar to WHERE but used after grouping
- Applies conditions to group by results

– Count students in each city where max marks > 90
SELECT city, COUNT(*)
FROM students
GROUP BY city
HAVING MAX(marks) > 90;

General Query Order

```
SELECT column(s)  
FROM table_name  
WHERE condition  
GROUP BY column(s)  
HAVING condition  
ORDER BY column(s) ASC;
```

Joins

What are Joins?

Joins combine rows from two or more tables based on related columns between them.

Types of Joins

1. Inner Join

- Returns records with matching values in both tables

```
SELECT column(s)  
FROM tableA  
INNER JOIN tableB  
ON tableA.col_name = tableB.col_name;
```

2. Left Join

- Returns all records from left table + matched records from right table

```
SELECT column(s)
FROM tableA
LEFT JOIN tableB
ON tableA.col_name = tableB.col_name;
```

3. Right Join

- Returns all records from right table + matched records from left table

```
SELECT column(s)
FROM tableA
RIGHT JOIN tableB
ON tableA.col_name = tableB.col_name;
```

4. Full Join (MySQL)

- Returns all records when there's a match in either table

– MySQL implementation using UNION

```
SELECT columns FROM tableA LEFT JOIN tableB ON condition
UNION
SELECT columns FROM tableA RIGHT JOIN tableB ON condition;
```

5. Self Join

- Table joined with itself

```
SELECT column(s)
FROM table AS a
JOIN table AS b
ON a.col_name = b.col_name;
```

Advanced Topics

UNION

- Combines result sets of two or more SELECT statements
- Returns unique records only

```
SELECT column(s) FROM tableA
UNION
SELECT column(s) FROM tableB;
```

Requirements for UNION:

- Same number of columns in each SELECT
- Similar data types
- Columns in same order

Sub Queries

- Query within another SQL query
- Also called Inner query or Nested query

```
SELECT column(s)  
FROM table_name  
WHERE col_name operator (subquery);
```

Example: Get students with marks above class average

```
SELECT name FROM students  
WHERE marks > (SELECT AVG(marks) FROM students);
```

Views

- Virtual table based on result of SQL statement
- Always shows up-to-date data
- Database recreates view each time it's queried

```
CREATE VIEW view_name AS  
SELECT column(s) FROM table_name WHERE condition;
```